

EXERCISE SOLUTIONS



# Module 7: NoSQL & Redis

Module 7 - 10 Hours

NoSQL & Redis

## Notes for the trainer

**Core and stretch.** The questions marked **(Core)** are the required set that fits the session. Debrief those with the whole room. Cover the unmarked stretch questions only if the batch worked ahead, or leave them as take-home reading.

Release after the practice window. Marks total 100.

| Section                     | Questions  | Marks |
|-----------------------------|------------|-------|
| Level 1 - Understanding     | Q1 to Q12  | 30    |
| Level 2 - Build and measure | Q13 to Q24 | 45    |
| Level 3 - Judgement         | Q25 to Q31 | 25    |

Measured numbers vary by machine. Mark the **ratio** and the **reasoning**, not the absolute figures.

# Level 1 - Understanding

## Q1 (Core)

NoSQL means "not only SQL": a data store that does not use the relational model, adopted where the data does not fit tables comfortably or does not fit on one machine.

| Family                    | ICICI Bank workload                                                         |
|---------------------------|-----------------------------------------------------------------------------|
| Key-value (Redis)         | Balance cache, sessions, rate limiting                                      |
| Document (MongoDB)        | KYC documents, where the set of documents varies per customer               |
| Column family (Cassandra) | Transaction history at very large scale, written once and read by key       |
| Graph (Neo4j)             | Fraud rings and beneficial ownership, where the relationships are the query |

## Q2 (Core)

**In one sentence:** a distributed system cannot simultaneously guarantee consistency, availability and tolerance of network partitions.

**Why "choose two of three" misleads:** partition tolerance is not a design option. Networks partition whether or not you planned for it, so a distributed system must tolerate partitions. The real choice arises only **during** a partition: answer with possibly stale data, or refuse to answer. Outside a partition you can have both C and A.

| System              | Gives up     | Why                                                                                                                    |
|---------------------|--------------|------------------------------------------------------------------------------------------------------------------------|
| Core banking ledger | Availability | Being wrong about money is worse than being unavailable. Refusing a transaction is recoverable; a wrong balance is not |
| Balance cache       | Consistency  | A balance a few seconds old is acceptable, and the database is always available as a fallback                          |
| Session store       | Consistency  | Asking a customer to log in again is an inconvenience, not an incident                                                 |

## Q3 (Core)

1. **Durability is best effort.** Even `appendfsync` `always` does not meet a ledger's standard, and `everysec` accepts up to a second of loss. A confirmed transfer that vanishes on restart is the Module 1 incident.
2. **Replication is asynchronous.** A failover promotes a replica that may be behind the primary, silently reverting the most recent writes. Two customers could see two different balances.
3. **No transactions across keys with rollback.** Redis has `MULTI` and Lua scripts, which are atomic, but there is no equivalent of a transaction that can be rolled back after a business rule fails partway. Debiting one account and crediting another cannot be made atomic across a failure the way Module 6 requires.

Also creditable: no constraints, so nothing prevents a negative balance; and eviction, which can silently discard a key if `maxmemory` is reached.

## Q4

**Advantage: atomicity for free.** `INCR` on a shared counter cannot lose an update, because no two commands ever interleave. A thousand application servers incrementing the same API counter need no coordination, and none of Module 6's locking applies.

**Danger: one slow command blocks everyone.** `KEYS *` on a cache with ten million keys stalls every client for seconds. During that stall every balance enquiry in the bank waits, and the connection pool fills.

## Q5 (Core)

| Requirement                    | Structure                      | Commands                                                                           |
|--------------------------------|--------------------------------|------------------------------------------------------------------------------------|
| Last ten transactions, capped  | List                           | <code>LPUSH</code> then <code>LTRIM key 0 9</code> , read with <code>LRANGE</code> |
| Is this card blocked           | Set                            | <code>SADD</code> , <code>SISMEMBER</code>                                         |
| Top three balances in a branch | Sorted set                     | <code>ZADD</code> , <code>ZREVRANGE key 0 2 WITHSCORES</code>                      |
| API call count today           | String counter                 | <code>INCR icici:api:hits:2026-03-11</code>                                        |
| Session with per-field update  | Hash                           | <code>HSET</code> , <code>HGET</code> , <code>EXPIRE</code>                        |
| Five OTPs per hour, sliding    | Sorted set scored by timestamp | <code>ZREMRANGEBYSCORE</code> , <code>ZCARD</code> , <code>ZADD</code>             |
| Nearest ATM within 2 km        | Geospatial                     | <code>GEOADD</code> , <code>GEOSEARCH ... BYRADIUS 2 km</code>                     |

## Q6 (Core)

|                          | Cache-aside             | Write-through  |
|--------------------------|-------------------------|----------------|
| Cache populated          | On a read miss          | On every write |
| Cost of a write          | Nothing extra           | A second write |
| First read after a write | A miss, then repopulate | A hit          |

Cache-aside only ever caches what somebody asked for, so the cache fills with genuinely hot data. Write-through caches everything written, including data nobody will read, and pays for it on every write.

## Q7

```
11:04:22 Application writes the new balance to Redis, returns "successful"
11:04:23 The Redis node fails before the flush to the database
11:04:24 The transfer is in no ledger and no log
```

The customer was told the transfer succeeded. The money did not move, there is no record that it was ever attempted, and reconciliation cannot detect it because there is nothing to reconcile against.

**Legitimate use:** page view counts, "last seen" timestamps, non-financial analytics counters. Anything where losing a few seconds of updates has no consequence and no audit obligation.

## Q8 (Core)

**Delete rather than overwrite.** Overwriting races with every other writer. If two transfers complete near-simultaneously and each writes its own computed balance into the cache, the loser of the race leaves a **wrong** value in place, with a fresh TTL, and no read will correct it until it expires. A delete is idempotent: whichever order the deletes land in, the next read rebuilds from the source of truth.

**Delete after the commit.** Deleting first opens a window between the delete and the commit. Another thread reading in that window queries the database, sees the **pre-commit** state, and re-caches the old value - which now survives the full TTL. The stale value is created by the invalidation itself.

## Q9 (Core)

The TTL is the backstop for **every invalidation that was never written or that failed to run**.

Correct invalidation in `transfer()` covers transfers made through that function. It does not cover a batch job, an overnight interest posting, a DBA correcting a row by hand, or a new service written next year by someone who does not know the cache exists. It also does not cover the case where the delete itself failed because Redis was briefly unreachable at that moment.

Without a TTL, any of those leaves a wrong value cached **forever**. With one, the damage is bounded to the TTL. That is why a cache key with `TTL = -1` is a defect regardless of how good the invalidation logic is.

## Q10 (Core)

**A stampede** is what happens when a hot key expires and many concurrent requests miss simultaneously, so hundreds of identical queries hit the database in the same millisecond. The cache makes the load spike worse than having no cache.

| Prevention    | How                                                                                         |
|---------------|---------------------------------------------------------------------------------------------|
| Jitter        | <code>TTL = base + random(0, 60)</code> so related keys do not expire together              |
| Lock on miss  | <code>SETNX</code> a lock; one request queries, the rest wait briefly and re-read the cache |
| Refresh ahead | Repopulate at around 80 percent of the TTL, before expiry                                   |

**Implement jitter first.** It is one line, it has no failure modes, and it addresses the common case of many keys cached together expiring together. The lock is worth adding afterwards for the handful of genuinely hot single keys, and refresh-ahead is the most complex and the most easily got wrong.

## Q11

With `maxmemory` unset, Redis grows until the operating system's out-of-memory killer terminates the process. **The entire cache is lost at once, without warning, at the moment of peak load** - because peak load is what filled the memory. Every request then misses, and the database receives the full unfiltered traffic at exactly the wrong time.

With a limit set and `allkeys-lru`, memory pressure causes gradual eviction of cold keys. Hit rate falls a little, database load rises a little, and there is time to notice and react.

**The volatile-lru trap:** it evicts only keys that have a TTL. If no key has one - or if the keys under pressure happen to be the ones without - Redis has nothing it is permitted to evict and writes start failing with an out-of-memory error, even though most of the data is discardable. The policy silently depends on a discipline that nothing enforces.

## Q12

| Possible cause | How to tell                                                                                    |
|----------------|------------------------------------------------------------------------------------------------|
| TTL too short  | Check <code>TTL</code> on live keys; compare the expiry interval with the request rate per key |

| Possible cause                 | How to tell                                                                                                                            |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Too many distinct keys         | <code>DBSIZE</code> against the number of distinct entities; a per-request key such as one including a timestamp gives near-zero reuse |
| Eviction under memory pressure | <code>INFO stats</code> and check <code>evicted_keys</code> ; if it is climbing, keys are being discarded before reuse                 |
| Over-aggressive invalidation   | Look for a write path deleting more keys than it changed, for example invalidating a whole branch on one account change                |

Distinguish them in that order, because the first two are design errors and the second two are operational. `DBSIZE` compared to the number of accounts is usually the fastest diagnostic: 36 accounts and 400,000 keys means the key design is wrong.

## Level 2 - Build and measure

### Q13 (Core)

```
DEL icici:ex:ministmt:9010
LPUSH icici:ex:ministmt:9010 '{"d":"2026-03-01","amt":-1200}'
LPUSH icici:ex:ministmt:9010 '{"d":"2026-03-02","amt":8000}'
LPUSH icici:ex:ministmt:9010 '{"d":"2026-03-03","amt":-450}'
LPUSH icici:ex:ministmt:9010 '{"d":"2026-03-04","amt":-2300}'
LPUSH icici:ex:ministmt:9010 '{"d":"2026-03-05","amt":15000}'
LTRIM icici:ex:ministmt:9010 0 4

LPUSH icici:ex:ministmt:9010 '{"d":"2026-03-06","amt":-99}'
LTRIM icici:ex:ministmt:9010 0 4
LRANGE icici:ex:ministmt:9010 0 -1
LLEN icici:ex:ministmt:9010
```

Five entries, newest first, and the 1 March entry is gone.

**The point to draw out:** `LTRIM` must follow **every** `LPUSH`, not run periodically. A cleanup job that trims once a minute allows the list to grow without bound in between, which on a busy account is exactly when memory matters. In `icici_cache.js` the two are issued in one `MULTI` so they travel together.

### Q14 (Core)

```
DEL icici:ex:branch5
ZADD icici:ex:branch5 91729.06 acct:9005 27637.82 acct:9006 276393.25 acct:9015 ...

ZREVRANGE icici:ex:branch5 0 1 WITHSCORES
ZREVRANK icici:ex:branch5 acct:9015
ZCOUNT icici:ex:branch5 200000 +inf
```

`ZREVRANK` rather than `ZRANK`, because the question is about rank by descending balance. `ZRANK` would give the position from the bottom, which is a quiet off-by-everything error.

`ZCOUNT` answers the third question without transferring any members, which is the advantage of a sorted set over doing it in the application.

### Q15

**Structure: a sorted set**, scored by timestamp.

```
async function recordView(rmId, accountId) {
  const key = `icici:ex:recent:${rmId}`;
  await r.multi()
    .zAdd(key, { score: Date.now() / 1000, value: String(accountId) })
    .zRemRangeByRank(key, 0, -6) // keep only the newest 5
    .expire(key, 86400)
    .exec();
}

const recentViews = (rmId) =>
  r.zRange(`icici:ex:recent:${rmId}`, 0, 4, { REV: true });
```

**Why a list does not work.** A list preserves order and can be capped with `LTRIM`, but it permits duplicates. A manager who views account 9003 five times would fill the list with five copies of it and push out everything else. Removing the existing entry first is not atomic and needs `LRM`, which is  $O(n)$ .

A sorted set gives both properties for free: adding an existing member **updates its score** rather than adding a duplicate, so the account simply moves to the front. `ZREMRANGEBYRANK key 0 -6` then caps the set at five.

This is the question that separates learners who have understood the structures from those who have memorised them.

## Q16 (Core)

```
async function getCustomerProfile(db, customerId) {
  const key = `icici:ex:profile:${customerId}`;

  const cached = await r.hGetAll(key);
  if (Object.keys(cached).length > 0) {
    M.hit();
    return [cached, 'cache'];
  }

  M.miss();
  const { rows } = await db.query(
    `SELECT first_name, last_name, city, kyc_status, risk_rating
     FROM customer WHERE customer_id = $1`, [customerId]);
  if (rows.length === 0) return [null, 'db'];

  const row = rows[0];
  const doc = {
    first_name: row.first_name, last_name: row.last_name,
    city: row.city ?? '', kyc_status: row.kyc_status,
    risk_rating: row.risk_rating ?? ''
  };
  await r.multi()
    .hSet(key, doc)
    .expire(key, 3600 + Math.floor(Math.random() * 300))
    .exec();
  return [doc, 'db'];
}
```

```
> await getCustomerProfile(db, 1003)
[ { first_name: 'Rohan', last_name: 'Iyer', city: 'Chennai',
  kyc_status: 'VERIFIED', risk_rating: 'HIGH' }, 'db' ]
> await getCustomerProfile(db, 1003)
[ { ... }, 'cache' ]
```

Note `row.city ?? ''`. `city` and `risk_rating` are nullable, and **Redis cannot store a NULL** - `hSet` with `null` or `undefined` throws, because there is no way to send it over the wire. Every cache over a table with nullable columns has to decide how it represents absence, and the decision must be reversible: an empty string and a genuine NULL are now indistinguishable, which may or may not matter. Learners who hit this error and worked around it silently should be asked what they lost.

## Q17 (Core)

```

async function getBranchSummary(db, branchId) {
  const key = `icici:ex:branchsum:${branchId}`;

  const cached = await r.hGetAll(key);
  if (Object.keys(cached).length > 0) {
    M.hit();
    return [cached, 'cache'];
  }

  M.miss();
  const { rows } = await db.query(
    `SELECT COUNT(DISTINCT c.customer_id) AS customers,
        COUNT(a.account_id)           AS accounts,
        COALESCE(SUM(a.balance), 0)   AS deposits
    FROM   customer c
        LEFT JOIN account a ON a.customer_id = c.customer_id
    WHERE  c.home_branch_id = $1`, [branchId]);

  const row = rows[0];
  const doc = {
    customers: String(row.customers),
    accounts:  String(row.accounts),
    deposits:  String(row.deposits),
  };
  await r.multi()
    .hSet(key, doc)
    .expire(key, 600 + Math.floor(Math.random() * 60))
    .exec();
  return [doc, 'db'];
}

```

Typical timings on the training dataset: cold around 3 ms, warm around 0.2 ms.

**Be honest about the scale.** On 36 accounts the aggregate is trivial and the cache saves almost nothing. The reason to cache it is that on 30,000 customers and 2 million transactions the same query takes seconds, and the cache is what makes the branch dashboard usable. Learners should not conclude from a 15x improvement on toy data that caching always pays.

`COUNT(DISTINCT c.customer_id)` is required for the Module 3 fan-out reason: a customer with two accounts would otherwise be counted twice.

## Q18

```

async function invalidateCustomer(db, customerId) {
  const keys = [`icici:ex:profile:${customerId}`];

  // ask the database which accounts belong to this customer
  const { rows } = await db.query(
    'SELECT account_id FROM account WHERE customer_id = $1', [customerId]);

  for (const { account_id: id } of rows) {
    keys.push(kBalance(id), kAccount(id), kMiniStatement(id));
  }

  if (keys.length) await r.unlink(keys);
  return keys.length;
}

```

**How the account keys are found:** by asking the database, which is the authority on which accounts belong to a customer. The alternative is to maintain a Redis set of the customer's account keys alongside the cache, which is faster but is itself a cached value that can go stale.

**Why `KEYS icici:*` is unacceptable:** it scans the entire keyspace and blocks the single-threaded server for the duration. On a production cache with millions of keys that is a multi-second stall across every client in the bank - an outage caused by an invalidation. If a pattern scan is genuinely needed, `SCAN` with a cursor is the only safe form, and even then it is a poor design compared with knowing which keys you need to delete.

## Q19 (Core)

```
const branches = [1, 2, 3, 4, 5, 6, 7, 8];
const pick = () => branches[Math.floor(Math.random() * branches.length)];

await r.flushDb(); M.reset();
let t0 = performance.now();
for (let i = 0; i < 500; i++) {
  await db.query(
    `SELECT COUNT(DISTINCT c.customer_id), COUNT(a.account_id),
      COALESCE(SUM(a.balance), 0)
    FROM customer c LEFT JOIN account a ON a.customer_id = c.customer_id
    WHERE c.home_branch_id = $1`, [pick()]);
}
console.log(`no cache: $${((performance.now() - t0) / 1000).toFixed(3)} s`);

await r.flushDb(); M.reset();
t0 = performance.now();
for (let i = 0; i < 500; i++) await getBranchSummary(db, pick());
console.log(`cached : $${((performance.now() - t0) / 1000).toFixed(3)} s`);
M.report();
```

Typical result:

```
no cache: 1.640 s
cached : 0.121 s
hits 492    misses 8    db calls 8    hit rate 98.4%
```

Eight misses, one per branch, exactly as expected: the miss count is bounded by the number of distinct keys, not by the number of requests.

## Q20 (Core)

```
console.log('1.', (await getCustomerProfile(db, 1003))[0].city, '(cached now)');

await db.query("UPDATE customer SET city = 'Coimbatore' WHERE customer_id = 1003");

const { rows } = await db.query(
  'SELECT city FROM customer WHERE customer_id = 1003');
console.log('2. database says:', rows[0].city);

console.log('3. cache says :', (await getCustomerProfile(db, 1003))[0].city);
```

1. Chennai (cached now)
2. database says: Coimbatore
3. cache says : Chennai

**The fix:**

```
async function updateCustomerCity(db, customerId, city) {
  await db.query('UPDATE customer SET city = $1 WHERE customer_id = $2',
    [city, customerId]); // commits first
  await r.unlink(`icici:ex:profile:${customerId}`); // then invalidate
}
```

```
await updateCustomerCity(db, 1003, 'Chennai');
console.log(await getCustomerProfile(db, 1003)); // ['Chennai', 'db'] - repopulated
```

Marking: full marks require the delete to come **after** the write has committed, and to be a delete rather than an `hSet` of the new value.

## Q21

```
async function getBranchPack(db) {
  const key = 'icici:ex:branchpack';
  const cached = await r.get(key);
  if (cached !== null) {
    M.hit();
    return [JSON.parse(cached), 'cache'];
  }

  M.miss();
  const { rows } = await db.query(`
    SELECT b.branch_name,
      COUNT(DISTINCT c.customer_id) AS customers,
      COUNT(DISTINCT a.account_id) AS accounts,
      COALESCE(SUM(CASE WHEN t.txn_type='CREDIT' THEN t.amount ELSE 0 END),0) AS credits,
      COALESCE(SUM(CASE WHEN t.txn_type='DEBIT' THEN t.amount ELSE 0 END),0) AS debits
    FROM branch b
      LEFT JOIN customer c ON c.home_branch_id = b.branch_id
      LEFT JOIN account a ON a.customer_id = c.customer_id
      LEFT JOIN account_txn t ON t.account_id = a.account_id
      AND t.status = 'SUCCESS'
    GROUP BY b.branch_name ORDER BY 4 DESC`);

  const pack = rows.map((row) => Object.values(row).map(String));
  await r.setEx(key, 900 + Math.floor(Math.random() * 120), JSON.stringify(pack));
  return [pack, 'db'];
}
```

Typical: cold around 14 ms, warm around 0.3 ms, a ratio near 45x on the training data and far larger on production volumes.

A JSON string is the right choice here rather than a hash, because the whole result is always read together and never partially updated. That is the inverse of the account-cache decision in Q16, and learners should be able to say why the two differ.

## Q22 (Core)

```
import { randomBytes } from 'node:crypto';

async function createSessionV2(customerId, device, ip, ttl = 1800) {
  const token = randomBytes(32).toString('base64url');
  const key = `icici:ex:session:${token}`;
  const now = new Date().toISOString();

  await r.multi()
    .hSet(key, {
      customer_id: String(customerId), device, ip,
      login_at: now, last_seen: now,
    })
    .expire(key, ttl)
    .sAdd(`icici:ex:sessions:${customerId}`, token)
    .expire(`icici:ex:sessions:${customerId}`, ttl * 2)
    .exec();
  return token;
}

async function activeSessions(customerId) {
  const out = [];
  for (const token of await r.sMembers(`icici:ex:sessions:${customerId}`)) {
    const key = `icici:ex:session:${token}`;
    const data = await r.hGetAll(key);
    if (Object.keys(data).length > 0) {
      data.ttl = await r.ttl(key);
      data.token = `${token.slice(0, 12)}...`;
      out.push(data);
    } else {
      // tidy the index - the session expired but its name did not
      await r.sRem(`icici:ex:sessions:${customerId}`, token);
    }
  }
  return out;
}
```

```
[{ customer_id:'1002', device:'ANDROID', ip:'10.2.3.4', ttl:1798, ... },
 { customer_id:'1002', device:'WEB',      ip:'10.2.3.9', ttl:1799, ... },
 { customer_id:'1002', device:'IOS',      ip:'10.2.4.1', ttl:1800, ... }]
```

**The detail worth marking:** the `else` branch removing dead tokens from the set. Sessions expire individually, but the set holding their names does not know that, so it accumulates references to keys that no longer exist. Without the cleanup, "sign out everywhere" iterates over hundreds of dead tokens and `activeSessions` returns a misleading count. **An index into expiring keys must be maintained**, and this is a bug that appears only after weeks in production.

## Q23 (Core)

```
async function recordFailedLogin(accountId, limit = 5, window = 900, lockFor = 1800) {
  const failKey = `icici:ex:login:fail:${accountId}`;
  const lockKey = `icici:ex:login:lock:${accountId}`;

  if (await r.exists(lockKey)) {
```

```

    return [false, 'LOCKED', await r.ttl(lockKey)];
  }

  const [fails] = await r.multi()
    .incr(failKey)
    .expire(failKey, window, 'NX') // arm once; a later failure cannot extend it
    .exec();

  if (Number(fails) >= limit) {
    await r.multi()
      .set(lockKey, '1', { EX: lockFor })
      .unlink(failKey)
      .exec();
    return [false, 'LOCKED', lockFor];
  }
  return [true, 'ALLOWED', Number(fails)];
}

const recordSuccessfulLogin = (accountId) =>
  r.unlink(`icici:ex:login:fail:${accountId}`);

```

```

attempt 1: ALLOWED (fails 1)
attempt 2: ALLOWED (fails 2)
attempt 3: ALLOWED (fails 3)
attempt 4: ALLOWED (fails 4)
attempt 5: LOCKED (locked for 1800s)
attempt 6: LOCKED (1794s remaining)

```

Two things to draw out. `recordSuccessfulLogin` must clear the counter, or a customer who mistypes their password three times over a week is eventually locked out for no reason. And the counter has its own `window` expiry, so failures age out - the limit is five failures **within fifteen minutes**, not five failures ever.

Ask the class where this control belongs. A Redis counter is fast and appropriate, but if Redis is unavailable the lockout disappears, which is a security control failing open. In a real bank the durable record of the lockout also lives in the database, with Redis as the fast path.

## Q24

```

const BLOCKED_KEY = 'icici:ex:blocked_cards';

async function loadBlockedCards(db) {
  const { rows } = await db.query(
    "SELECT card_id FROM card WHERE status = 'BLOCKED'");
  const ids = rows.map((row) => String(row.card_id));

  const m = r.multi().unlink(BLOCKED_KEY);
  if (ids.length) m.sAdd(BLOCKED_KEY, ids);
  await m.exec();
  return ids.length;
}

async function blockCard(db, cardId) {
  await db.query("UPDATE card SET status = 'BLOCKED' WHERE card_id = $1", [cardId]);
  await r.sAdd(BLOCKED_KEY, String(cardId));
}

```

```
async function isCardBlocked(db, cardId) {
  try {
    return await r.sIsMember(BLOCKED_KEY, String(cardId));
  } catch {
    // fail CLOSED: Redis is unavailable, so ask the database
    const { rows } = await db.query(
      'SELECT status FROM card WHERE card_id = $1', [cardId]);
    return rows.length > 0 && rows[0].status === 'BLOCKED';
  }
}
```

If **Redis is unavailable**, the code falls back to the database. That is slower, and it is the only acceptable behaviour: this is a **security control**, so it must fail closed.

Contrast with the balance cache, where a Redis failure falls back to the database and the customer sees a slightly slower screen. Here, returning `False` on error - "not blocked" - would authorise transactions on cards the bank has blocked, which is a fraud loss rather than a performance problem.

Marking: full marks require the fallback and an explicit statement about failing closed. A `try/catch` returning `false` should score zero for this part, and is worth discussing with the whole class.

## Level 3 - Judgement

### Q25 (Core)

| Item                               | Cache? | TTL or reason                                                                                             |
|------------------------------------|--------|-----------------------------------------------------------------------------------------------------------|
| Account balance                    | Yes    | 60 to 300 s. Read constantly, changes rarely, brief staleness tolerable                                   |
| Result of a fund transfer          | No     | It is a write, not a read. Nothing re-reads it, and caching a write result invites write-behind           |
| Branch and IFSC list               | Yes    | 24 hours. Reference data that changes a few times a year                                                  |
| USD to INR rate                    | Yes    | 30 to 60 s. Read constantly; staleness has a direct financial cost, so keep it short                      |
| Seven years of transaction history | No     | Large, read rarely, and every request is for a different range. Near-zero hit rate at high memory cost    |
| Whether a card is blocked          | Yes    | No TTL, or a long one, with explicit invalidation on block. A security control; see Q24 on failing closed |
| A one-time password just sent      | Yes    | It <b>belongs</b> in Redis with a 5-minute TTL. The expiry is the security requirement                    |
| Failed login attempt count         | Yes    | 15-minute window. Same reasoning as the OTP                                                               |

The distinction to draw out: the last three are not caches at all. **They are data that lives in Redis**, where expiry is the feature rather than a safety net. A cache holds a copy of something authoritative elsewhere; an OTP has no other home.

### Q26 (Core)

#### Model answer.

The 99.9 percent hit rate is real, and it is not the number that matters.

A 24-hour TTL means a customer can be shown a balance that is up to a day out of date. They make a payment at 09:00, the money leaves the account, and the app keeps showing the old figure until the next morning. They will believe they have funds they do not have, attempt another payment, and call the contact centre. The cache would have converted a database load problem into a customer trust problem.

**What I would ask them:** does the balance screen invalidate the cache when the customer transacts through the app? If it does, most of the staleness disappears and the long TTL becomes far less dangerous. But it will not catch a salary credit, a standing instruction, an ATM withdrawal on another channel, or an interest posting - so the TTL remains the only bound on how wrong the display can be.

**What I would propose:** 300 seconds, with jitter, plus explicit invalidation on every write path we control. That gives us a measured 96 to 98 percent hit rate and removes 98 percent of database calls, which achieves everything the 24-hour TTL was reaching for. The last two percentage points of hit rate are not worth a day of staleness.

**How to decide the number:** it is not an engineering decision. The question for the product owner is "how out of date may a displayed balance be before we would consider it a customer-facing defect?" Whatever they answer is the TTL. If they say "never", the answer is that we cannot cache it at all, and we should have that conversation explicitly rather than picking a number ourselves.

## Q27 (Core)

| Component        | What happens                                                                                                                  | Acceptable? | What I would change                                                                                                                                                        |
|------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Balance endpoint | <code>r.get</code> raises a connection error. In the code as written, the exception propagates and the endpoint returns a 500 | No          | Wrap the cache read in <code>try/except</code> , log, and fall through to the database. A cache failure must degrade to slow, never to broken                              |
| Session store    | Every session is gone. Every customer is logged out and must sign in again                                                    | Partly      | Unavoidable if sessions live only in Redis, and it is why Redis persistence should be enabled. Sentinel with a replica turns a total loss into a few seconds of disruption |
| Rate limiter     | The counters are gone. Limits reset, and with the code as written the <code>try</code> block may fail open                    | No          | Fail closed for security controls, per Q24. The durable record belongs in the database with Redis as the fast path                                                         |

The general principle: **a cache failure should be a performance event; a security control failure must not be a bypass.** The same Redis outage must produce different behaviour in different components, and that has to be written deliberately - the default behaviour of an unhandled exception is the same everywhere, and it is wrong in at least one of these cases.

## Q28

### Model answer.

This would make Redis the system of record for account balances, and I cannot agree to it. Here is specifically what we would lose.

**Durability.** Redis acknowledges a write before it is safely stored. Even at the strictest setting it accepts up to a second of loss on a crash, and with a nightly flush we would be accepting up to twenty-four hours. A balance change confirmed to a customer could simply not exist after a restart, with nothing in any log to say it ever happened.

**Transactions.** A transfer must debit one account and credit another as one indivisible operation that can be rolled back if a rule fails partway. Redis has atomic commands and scripts, but it has no equivalent of the transaction Module 6 relies on. We would be rebuilding, in application code, the guarantee the database already provides - and getting it wrong, because everybody does.

**Constraints.** The database refuses a negative balance, an account with no customer, and a transaction type that is not a debit or a credit. Redis will store whatever bytes it is given. Every rule we have declared once would become a rule every application has to remember.

**Eviction.** Under memory pressure Redis discards keys. A balance is a key. There is no configuration in which "the customer's balance was evicted" is an acceptable sentence.

**Audit.** The regulator requires us to reconstruct the bank's position at a point in time. A nightly snapshot cannot answer what a balance was at 14:22 on a Tuesday.

The premise is also worth challenging. After the caching work, the database is not "just for reporting" - it is handling 130 balance reads a minute instead of 4,000 **because** the cache is in front of it. The database being quiet is evidence the design is working, not evidence it is unnecessary.

What I would support is extending the cache: more entity types, longer TTLs where the business agrees, and a read replica for reporting. All of that keeps the ledger where it belongs.

## Q29

### Three other paths that change a balance:

1. **The overnight interest posting batch**, which credits interest to every savings account directly in SQL.
2. **A DBA correction**, applied by hand during an incident to fix a mis-posted transaction.
3. **Another service** - a loan disbursement, a standing instruction processor, or a channel written by a different team that calls `sp_fund_transfer` without knowing a cache exists.

**An architecture that catches all of them: change data capture.** The database's own write-ahead log is the one thing every path goes through. A CDC connector reads the log, publishes a change event for every committed row change to `account`, and a small consumer deletes the corresponding cache key.

```
Any writer -> PostgreSQL -> WAL -> CDC connector -> event -> cache invalidator -> DEL key
```

Nothing can change a balance without producing a WAL record, so nothing can bypass the invalidation. It also invalidates **after** the commit by construction, because the log record only exists once the change is committed.

**The cost:** another moving part to run and monitor, replication slots that will fill the disk if the consumer stops, additional latency of tens to hundreds of milliseconds between the commit and the invalidation, and a new failure mode where the consumer silently falls behind. It is worth it in a bank with many writers and a strict staleness requirement. For a smaller system, a shorter TTL is the cheaper answer to the same problem.

This is the same mechanism Module 11 uses for incremental migration, and Module 8's outbox pattern is the simpler cousin of it.

## Q30

**Answer: no, and pre-loading would probably make things worse.**

The remaining 4 percent are not waste. They are the **first** request for each key: the cold start after a deploy or a restart, an account that has not been touched recently, and the request immediately after an invalidation - which is a miss by design and is exactly what keeps the cache correct.

**What pre-loading would cost.** Loading every account at startup means one large query over the whole table at the moment the service starts, which is when it is least able to absorb it. Memory would hold every account rather than the hot ones, so `maxmemory` has to grow to fit the entire dataset instead of the working set. And a full pre-load makes the cache stale on a much larger scale, because entries that nobody reads still occupy memory and still expire and reload on a schedule nobody is watching.

There is also a smaller point worth making: at 96 percent, the remaining database load is around 130 calls a minute. Eliminating it entirely saves 130 calls a minute on a system that comfortably handled 4,000. **There is no problem left to solve.**

**When pre-loading is the right choice:** a small, bounded, universally-read reference set. The branch and IFSC list is a good candidate - a few hundred rows, read on nearly every screen, changing a few times a year. Loading it at startup means no user ever pays for the miss, and the memory cost is trivial. The technique is right; the balance table is the wrong place for it.

## Q31

**Model answer.**

Dear Ms Krishnan,

Balance enquiries now reach the core banking database about 130 times a minute, against 4,000 before.

Roughly 97 of every 100 enquiries are answered without touching the core system at all, and the response the customer sees is faster as well.

The way it works is simple. When someone asks for a balance, we answer from a small, very fast store that keeps a copy of recent answers. If we do not have a copy, we ask the core system as before and keep the answer for a few minutes. Because most customers check their balance several times in a session, and because a balance usually does not change between those checks, the same answer serves many enquiries.

The obvious question is whether a kept copy can be out of date. It can, and we have bounded it in two ways.

Whenever money moves through our own systems we discard the copy immediately, so the very next enquiry gets a fresh figure from the core. And every copy is discarded automatically after a few minutes whether or not anything happened, so even a change we did not see about corrects itself quickly.

The thing that could go wrong is a balance changing through a route we do not currently watch, such as an overnight batch posting. In that window a customer could see a figure a few minutes old. That is why the automatic discard exists, and why the interval is short rather than long. If you would prefer it shorter still, we can do that; it costs a little more load on the core system and I would rather you set the number than I do.

On internet banking sessions: these have moved off the MySQL database into the same fast store. Session checks are no longer the slowest part of loading a page, expired sessions are removed automatically rather than by a nightly job, and we can now sign a customer out of every device at once, which we could not do before.

Marking: 5 marks. Look for the measured figure, the copy explained without jargon, both bounds on staleness described, an honest named risk with its mitigation, the TTL offered as the business's decision, and the session improvement including the capability that is new rather than only faster.